

Experiment: Breadth First Search (BFS)

Aim

To implement the Breadth First Search (BFS) algorithm using Python and visualize the graph traversal.

Theory

Breadth First Search (BFS) is an uninformed search algorithm that traverses a graph level by level. It starts from a source node, visits all its neighboring nodes, and then proceeds to the next level. BFS uses a **Queue (FIFO)** data structure to keep track of the nodes that need to be explored.

Algorithm

1. Create an empty graph.
2. Add the required edges to the graph.
3. Select the starting node.
4. Initialize an empty list named `visited`.
5. Initialize a queue and insert the starting node.
6. Repeat until the queue becomes empty:
 - (a) Remove the front node from the queue.
 - (b) If the node is not visited:
 - i. Mark it as visited.
 - ii. Insert all unvisited neighboring nodes into the queue.
7. Display the BFS traversal.
8. Draw the graph using NetworkX and Matplotlib.

Python Program

```
1
2 import networkx as nx
3
4 # NetworkX is a Python package used to create,
5 # manipulate, analyze, and visualize graphs and networks.
6
7 import matplotlib.pyplot as plt
8
9 from collections import deque
10
11 # deque (Double-Ended Queue) is imported from
12 # Python's built-in collections module.
13 # BFS follows FIFO (First In First Out) principle.
14 # The first node inserted into the queue is
15 # the first node removed.
16
17 # Create Graph
18 G = nx.Graph() # Creates an empty graph
19
20 edges = [
21     ('A', 'B'), ('A', 'C'),
22     ('B', 'D'), ('B', 'E'),
23     ('C', 'F'), ('C', 'G'),
24     ('E', 'H')
25 ]
26
27 G.add_edges_from(edges)
28
29
30 # Breadth First Search (BFS)
31
32 start = 'A'
33
34 visited = []
35
36 queue = deque([start])
37
38
39 while queue:
40
```

```

41     node = queue.popleft()
42
43     if node not in visited:
44
45         visited.append(node)
46
47         queue.extend(
48             sorted(set(G.neighbors(node))
49                   - set(visited))
50         )
51
52
53 print("BFS Traversal:", visited)
54
55
56 # Draw Graph
57
58 pos = nx.spring_layout(G, seed=42)
59
60
61 node_colors = [
62     'red' if node in visited
63     else 'lightblue'
64     for node in G.nodes()
65 ]
66
67
68 nx.draw(
69     G,
70     pos,
71     with_labels=True,
72     node_size=1000,
73     node_color=node_colors,
74     font_size=12
75 )
76
77
78 plt.title("Breadth First Search")
79
80 plt.show()

```

Explanation

- `import networkx as nx` imports the NetworkX library for creating and manipulating graphs.
- `import matplotlib.pyplot as plt` imports Matplotlib for graph visualization.
- `from collections import deque` imports the **deque** data structure, which efficiently implements the queue required for BFS.
- `G = nx.Graph()` creates an empty undirected graph.
- `G.add_edges_from(edges)` adds all the specified edges to the graph.
- The variable `start` stores the source node from which the traversal begins.
- The list `visited` keeps track of all visited nodes.
- `queue = deque([start])` initializes the queue with the starting node.
- The `while` loop continues until the queue becomes empty.
- `queue.popleft()` removes the first node from the queue, following the FIFO principle.
- If the node has not been visited, it is added to the `visited` list.
- `G.neighbors(node)` returns all adjacent nodes of the current node.
- The expression

```
sorted(set(G.neighbors(node)) - set(visited))
```

returns all unvisited neighboring nodes in sorted order.

- `queue.extend()` inserts these neighboring nodes at the end of the queue.
- After completing the traversal, the visited nodes are printed as the BFS traversal order.
- `nx.spring_layout()` automatically calculates suitable positions for displaying the graph.
- A list comprehension assigns colors to the nodes:
 - Red indicates visited nodes.
 - Light blue indicates unvisited nodes.

- `nx.draw()` displays the graph with node labels and colors.
- `plt.show()` displays the graph window.
- `nx.spring_layout()` is a NetworkX function used for graph visualization. It automatically calculates suitable positions for each node so that the graph appears clear and well organized.
- The parameter `seed=42` ensures that the same node arrangement is generated every time the program is executed.

Output

BFS Traversal:

```
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
```

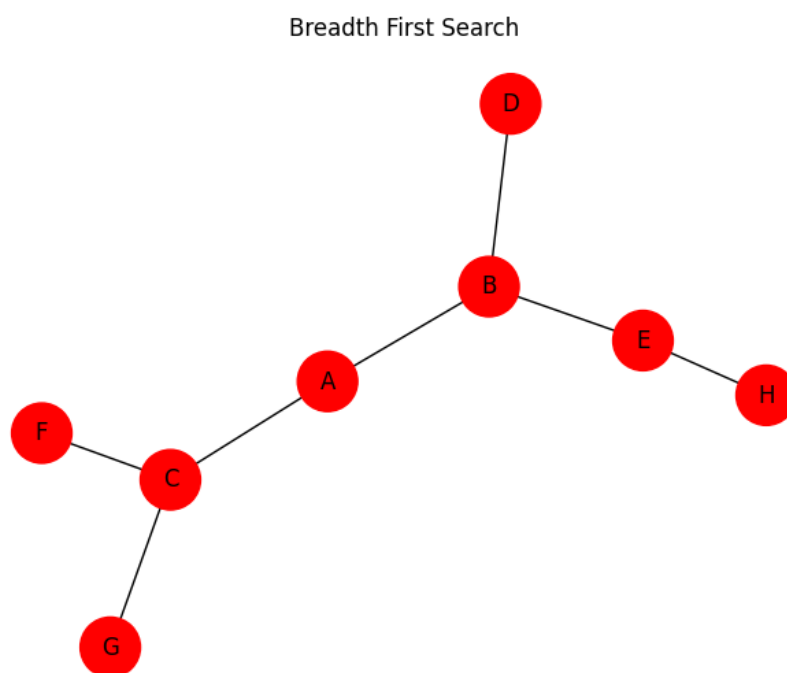


Figure 1: Graphical Visualization of BFS Traversal

Result

The Breadth First Search (BFS) algorithm was successfully implemented using Python. The graph was traversed level by level using a queue, and the traversal order along with its graphical visualization was obtained successfully.

Time Complexity of Breadth First Search (BFS)

The time complexity of Breadth First Search (BFS) depends on the representation of the graph.

Let:

- V = Number of vertices (nodes)
- E = Number of edges

BFS Using Adjacency List

In an adjacency list representation:

- Each vertex is visited exactly once.
- Each edge is explored exactly once.

Therefore, the time complexity is:

$$\boxed{\text{Time Complexity} = O(V + E)}$$

Explanation:

BFS performs the following operations:

- Visiting all vertices requires $O(V)$ time.
- Checking all edges requires $O(E)$ time.

Hence,

$$O(V) + O(E) = O(V + E)$$

Program for animated visualisation of BFS

Python Program

The following Python program demonstrates the implementation of Breadth First Search (BFS) using an animated graph. The graph traversal is visualized using NetworkX and Matplotlib animation.

```
1 # Import required libraries
2
3 import networkx as nx
4 import matplotlib.pyplot as plt
5
6 from matplotlib.animation import FuncAnimation
7 from collections import deque
8
9 from IPython.display import HTML
10
11
12 # Create graph
13
14 G = nx.Graph()
15
16
17 edges = [
18     ('A', 'B'),
19     ('A', 'C'),
20     ('B', 'D'),
21     ('B', 'E'),
22     ('C', 'F'),
23     ('C', 'G'),
24     ('E', 'H')
25 ]
26
27
28 G.add_edges_from(edges)
29
30
31 # BFS Traversal
32
33 visited = []
34 parent = {}
```

```

35
36 queue = deque(['A'])
37
38
39 while queue:
40
41     node = queue.popleft()
42
43     if node not in visited:
44
45         visited.append(node)
46
47         for neighbour in sorted(G.neighbors(node)):
48
49             if neighbour not in visited and neighbour not in
queue:
50
51                 parent[neighbour] = node
52                 queue.append(neighbour)
53
54
55
56 print("BFS Traversal Order:")
57 print(visited)
58
59
60
61 # Graph Layout
62
63 pos = nx.spring_layout(G, seed=5)
64
65
66 fig, ax = plt.subplots(figsize=(8,6))
67
68
69 # Animation Function
70
71 def update(frame):
72
73     ax.clear()
74

```

```

75     visited_now = visited[:frame+1]
76
77
78     node_colors = [
79         'red' if n in visited_now else 'lightgray'
80         for n in G.nodes()
81     ]
82
83
84     edge_colors = []
85
86
87     for u,v in G.edges():
88
89         if ((v in parent and parent[v] == u and v in visited_now)
90             or
91             (u in parent and parent[u] == v and u in visited_now)
92         ):
93
94             edge_colors.append('green')
95
96         else:
97
98             edge_colors.append('black')
99
100     nx.draw(
101         G,
102         pos,
103         ax=ax,
104         with_labels=True,
105         node_color=node_colors,
106         edge_color=edge_colors,
107         node_size=1000,
108         width=3
109     )
110
111     ax.set_title(
112         f"Current Node : {visited[frame]}"
113     )

```

```

114
115
116
117 # Generate Animation
118
119 ani = FuncAnimation(
120     fig,
121     update,
122     frames=len(visited),
123     interval=1200,
124     repeat=False
125 )
126
127
128 # Display animation
129
130 HTML(ani.to_jshtml())

```

Sample Output

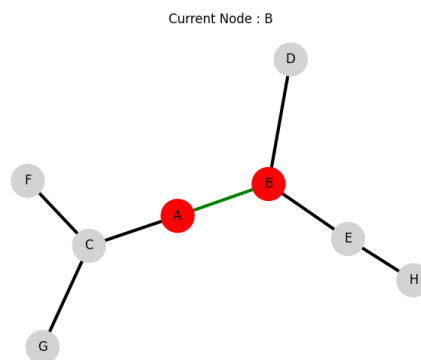


Figure 2: One of the animated traversal in BFS

Complexity Analysis

Let:

- V = Number of vertices
- E = Number of edges

For graph representation using an adjacency list:

$$\boxed{\text{Time Complexity} = O(V + E)}$$

Result

The BFS algorithm explores nodes level by level using a queue data structure. The animation demonstrates how nodes are expanded systematically and how the BFS search tree is generated during traversal.